

Week-4

Azure Cloud Fundamentals and Data Pipeline Implementation using ADF

Objective The primary objective of this project was to understand foundational Azure cloud concepts and apply them practically by building a secure, end-to-end data pipeline. The project required the integration of Azure Storage Accounts for data hosting and Azure Data Factory (ADF) for orchestration and data movement, while adhering to cloud security best practices through Identity and Access Management (IAM).

Phase 1: Cloud Infrastructure and Storage Provisioning The project began by establishing the core cloud infrastructure. A dedicated Resource Group (superstore-pipeline) was created to serve as the logical management container for all associated services. Within this group, an Azure Storage Account (stsuperstoredata123) was deployed. To facilitate the ETL (Extract, Transform, Load) process, the storage account was segmented into two distinct Blob Containers: input-data to act as the source repository and output-data to serve as the destination. A raw Sample - Superstore.csv dataset was manually uploaded into the input-data container to simulate incoming source data.

Phase 2: Security and Access Management (IAM) Before establishing connections, security protocols were implemented. An Azure Data Factory instance (adf-superstore-etl) was provisioned with a System-assigned Managed Identity. To ensure ADF had the necessary permissions to read from the source and write to the destination, Role-Based Access Control (RBAC) was utilized. The Data Factory's Managed Identity was officially assigned the "Storage Blob Data Contributor" role at the Storage Account level. This eliminated the need to use hardcoded access keys and ensured a secure, authenticated connection between the services.

Phase 3: Data Factory Configuration and Connectivity Within the Azure Data Factory Studio, the architectural components of the pipeline were configured. A Linked Service (LS_AzureBlobStorage1) was created to act as the official connection string linking ADF to the Storage Account. Building upon this linked service, two specific Datasets were defined:

- **Source Dataset (DS_Source):** Mapped precisely to read the Sample - Superstore.csv file located in the input-data container.
- **Destination Dataset (DS_Destination):** Mapped to target the output-data container where the final data would be deposited.

Phase 4: Pipeline Development and Orchestration With the datasets established, the visual pipeline was constructed. To ensure data integrity, a "Get Metadata" activity was placed at the beginning of the pipeline. This activity was configured with an "Exists" argument to programmatically verify that the source CSV file was actually present in the input-data container before attempting any operations. A success condition (green arrow) was then used to link the "Get Metadata" activity to a "Copy Data" activity. The Copy Data activity was configured to use DS_Source as its input and DS_Destination as its sink.

Phase 5: Execution and Validation The fully constructed pipeline was executed using the ADF Debug feature. The pipeline ran sequentially: the "Get Metadata" activity successfully

validated the presence of the CSV file, which subsequently triggered the "Copy Data" activity. Both activities returned a "Succeeded" status in the execution output logs. Final verification within the Azure Portal confirmed that the CSV data had been successfully copied from the source container into the output-data destination container, proving the end-to-end data pipeline was fully functional.

Task 1: Explore Azure Portal

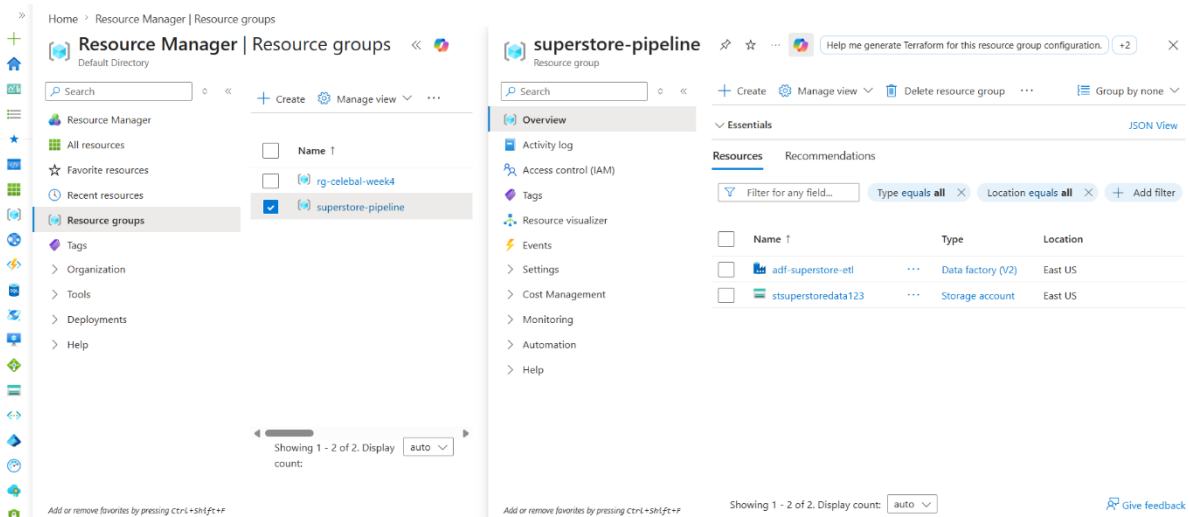


Fig 1: Screenshot showing the **successful creation of the superstore-pipeline** Resource Group in the Azure Portal, which acts as the logical container for this project.

Task 2: Storage Setup

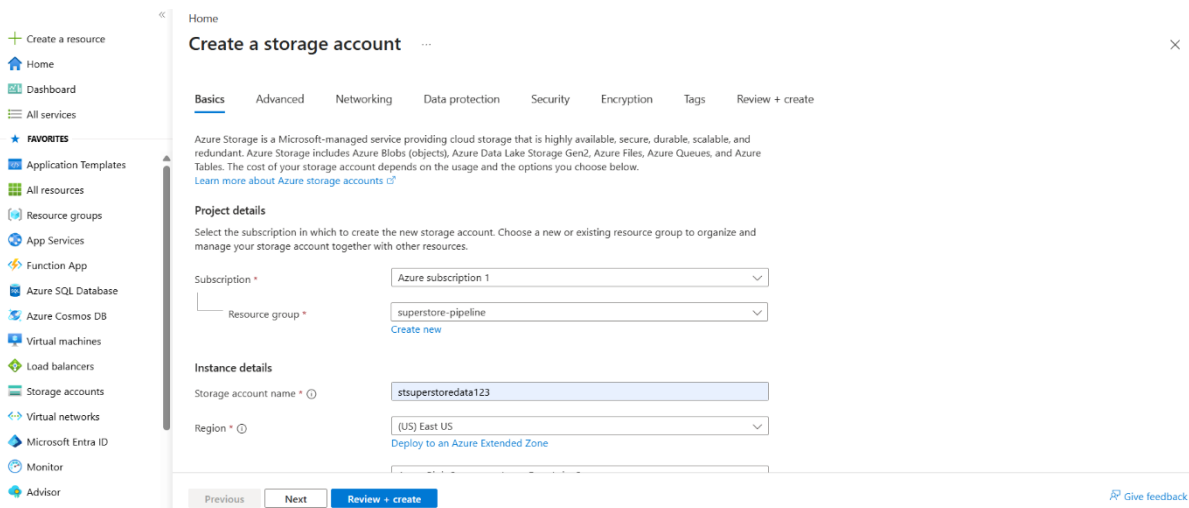


Fig 2: Screenshot showing the **successful creation and listing of the (stsuperstoredata123)** Resource Group in the Azure Portal, which serves as the foundational logical container for all subsequent pipeline resources.

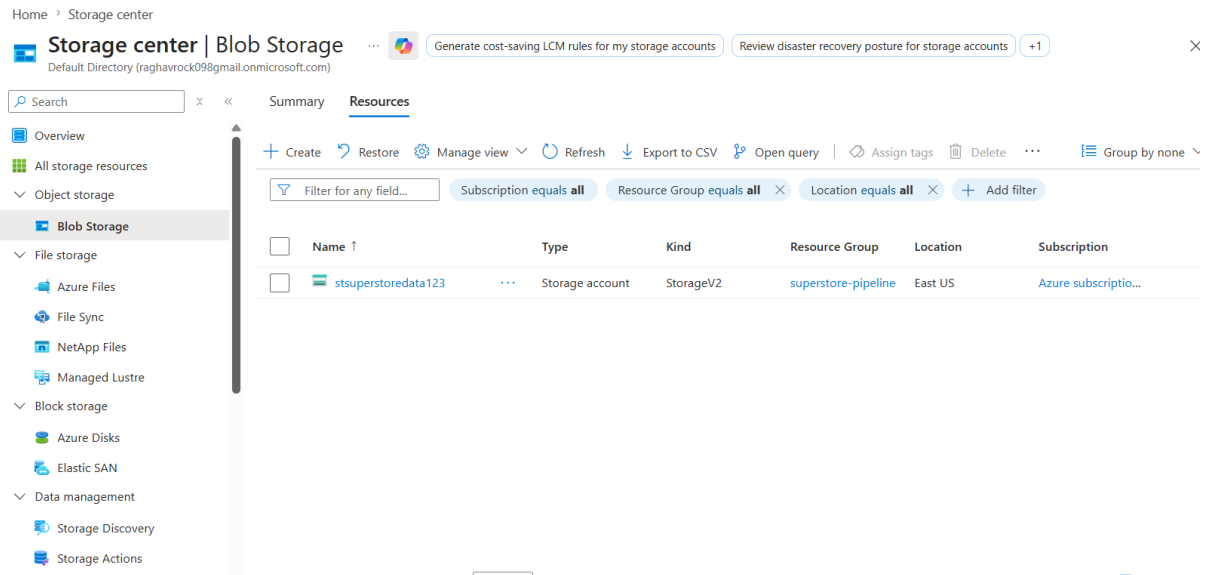


Fig 3: Successful creation and deployment of the Azure Storage Account (stsuperstoredata123) configured for Blob Storage within the designated resource group.

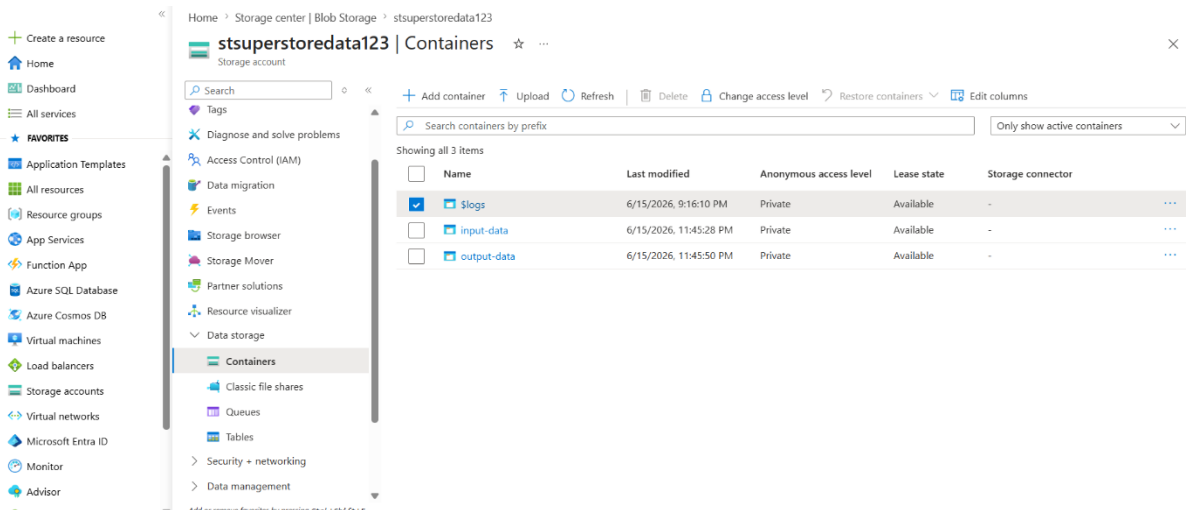


Fig 4: Displaying the successful creation of the input-data (source) and output-data (destination) Blob Containers within the Azure Storage Account, establishing the locations for data ingestion and pipeline output.

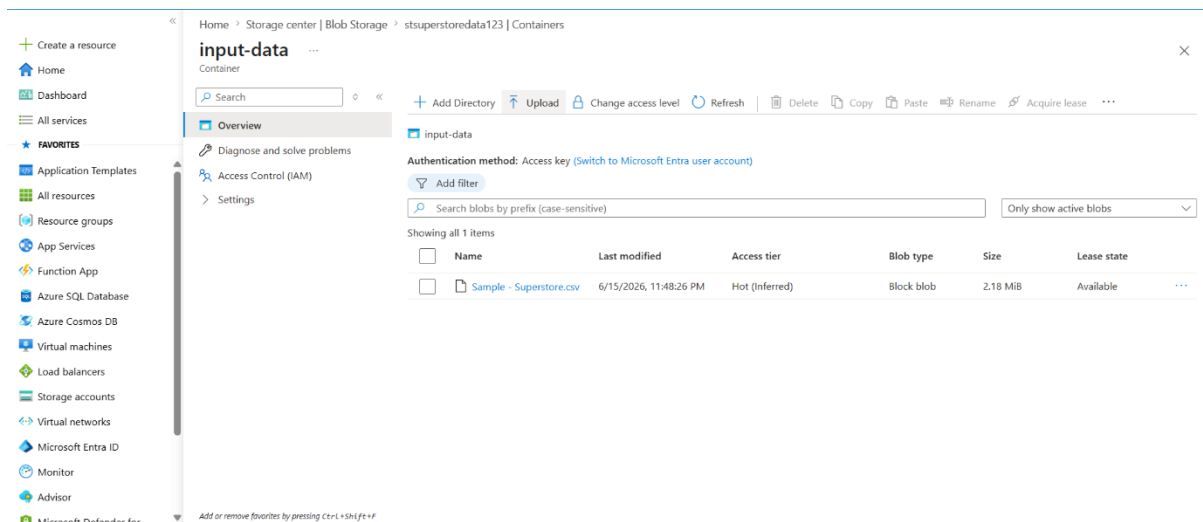


Fig 5: Screenshot verifying the successful upload of the raw Sample - Superstore.csv dataset into the input-data source container in Blob Storage.

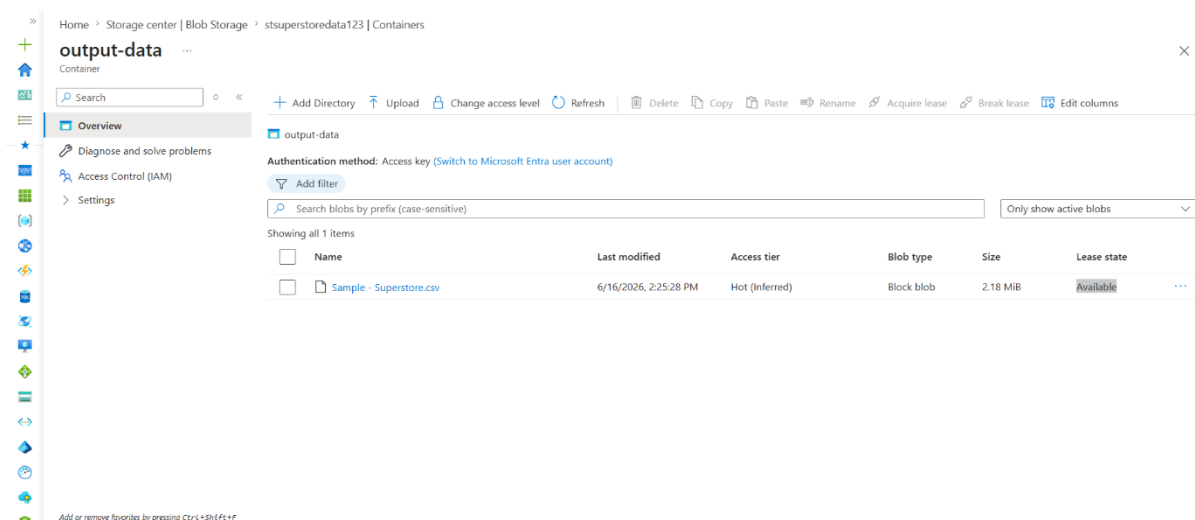


Fig 6: Screenshot of the output-data destination container, proving that the Azure Data Factory pipeline successfully executed and copied the CSV file to the target location.

Task 3: ADF Basics

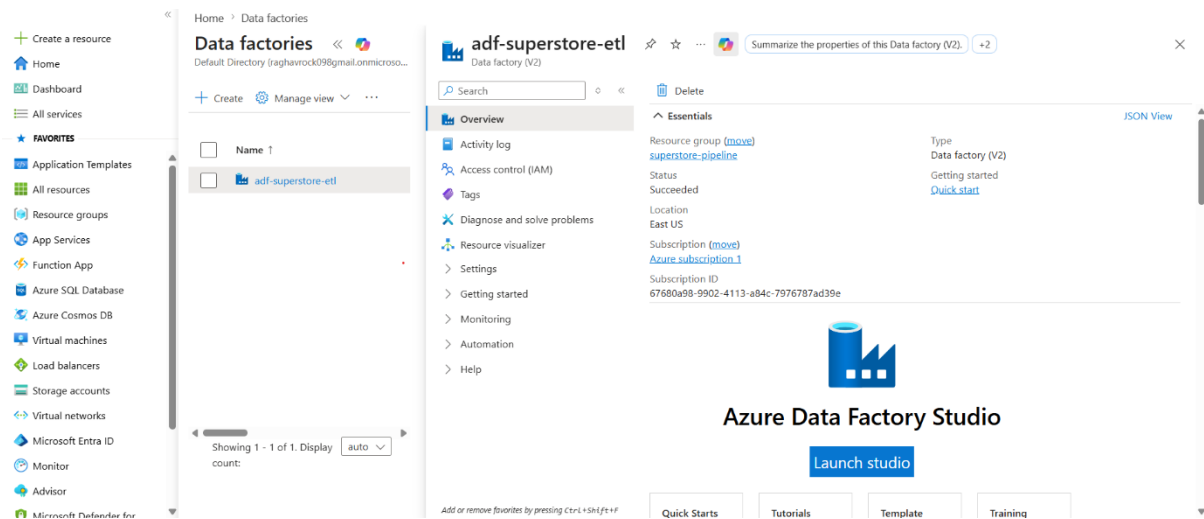


Fig 7: The Azure portal overview page confirming the successful creation and deployment of the Azure Data Factory resource (adf-superstore-etl) within the designated resource group, ready for pipeline development.

Task 3.1: ADF Basics (Linked Service)

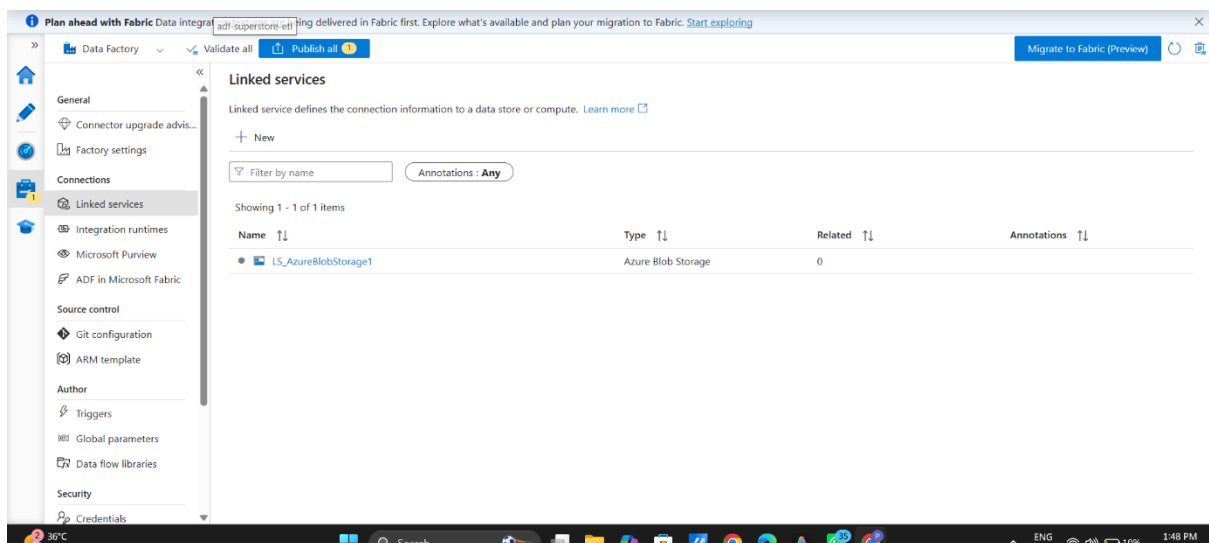


Fig 8: The Linked Service (LS_AzureBlobStorage1) is configured within Azure Data Factory to establish a secure, authenticated connection to the underlying Blob Storage account, utilizing a System-assigned Managed Identity for access control.

Task 3.2: ADF Basics (Datasets)

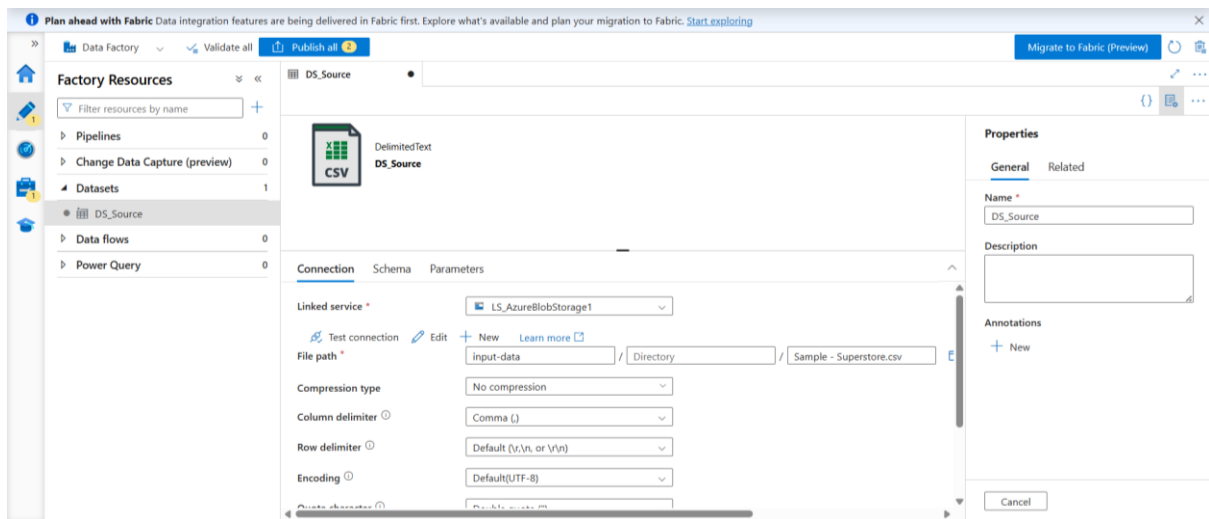


Fig 9: The **source dataset (DS_Source)** is actively configured to read the incoming Sample - Superstore.csv file directly from the input-data Blob container.

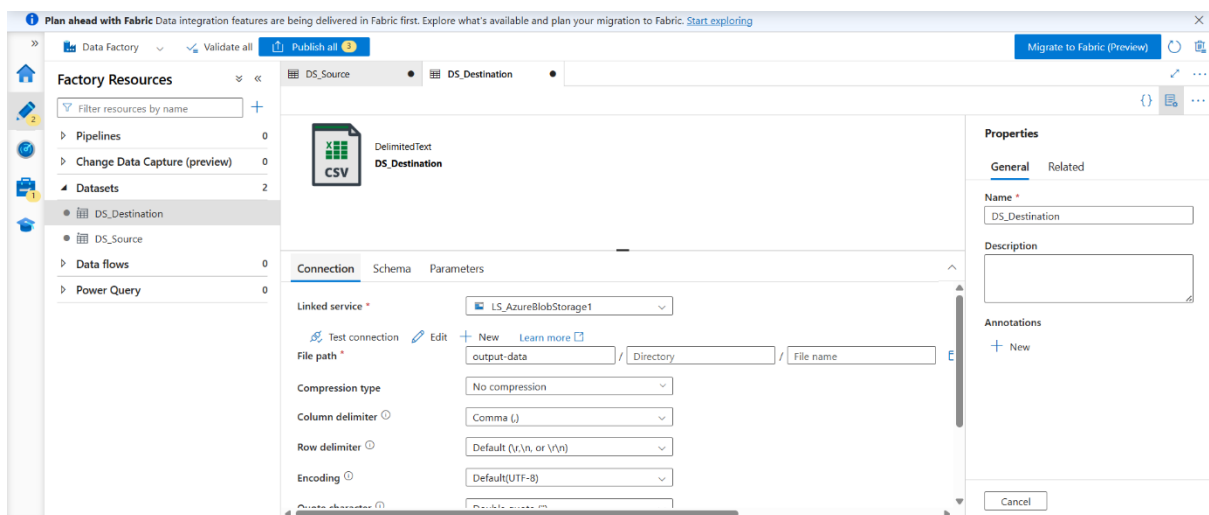


Fig 10: The **destination dataset (DS_Destination)** is mapped to the output-data Blob container, establishing the exact target location where the pipeline will write the final data.

Task 3.3: ADF Basics (Get Metadata)

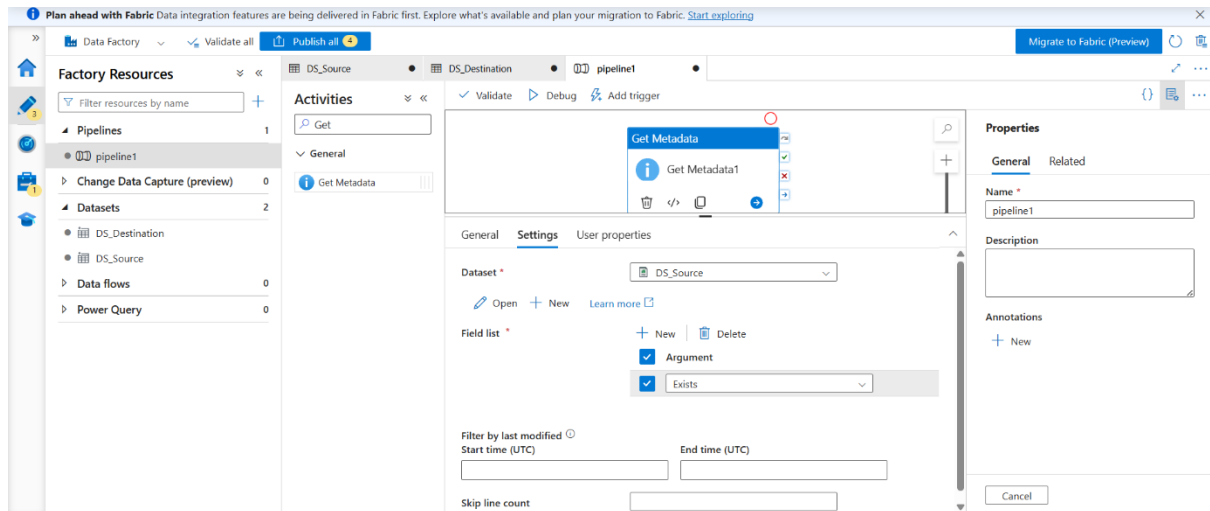


Fig 11: The "Get Metadata" pipeline activity is configured with the "Exists" argument, allowing it to validate the presence of the source file in Blob Storage before any data movement occurs.

Task 4: Pipeline Development

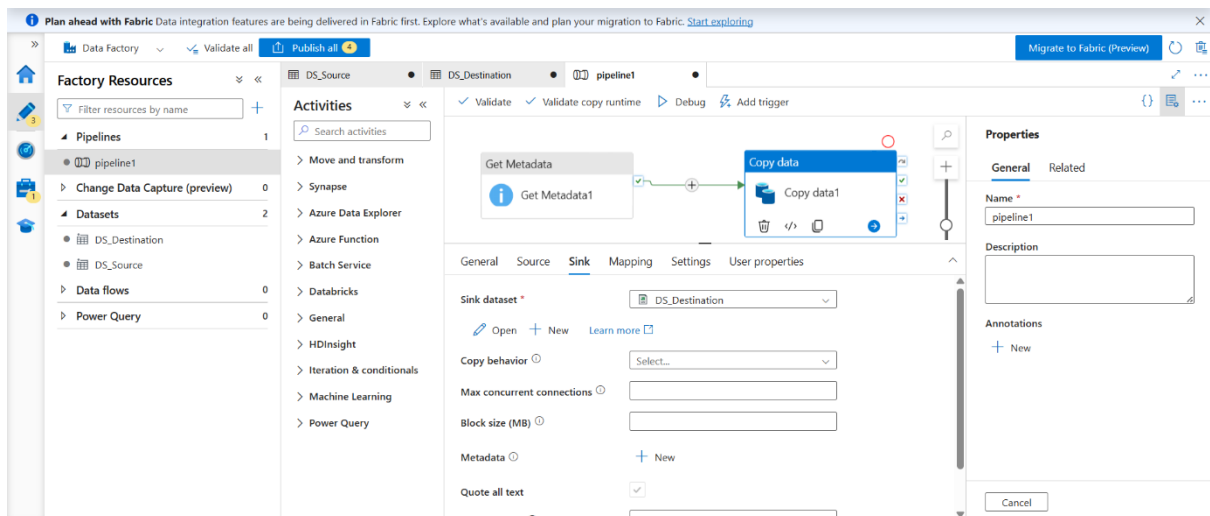


Fig 12: The complete pipeline design features a logical workflow where the "Get Metadata" validation step is connected via a success condition to the "Copy Data" activity, which is configured to write the final output to the destination dataset.

Task 5: Pipeline Execution

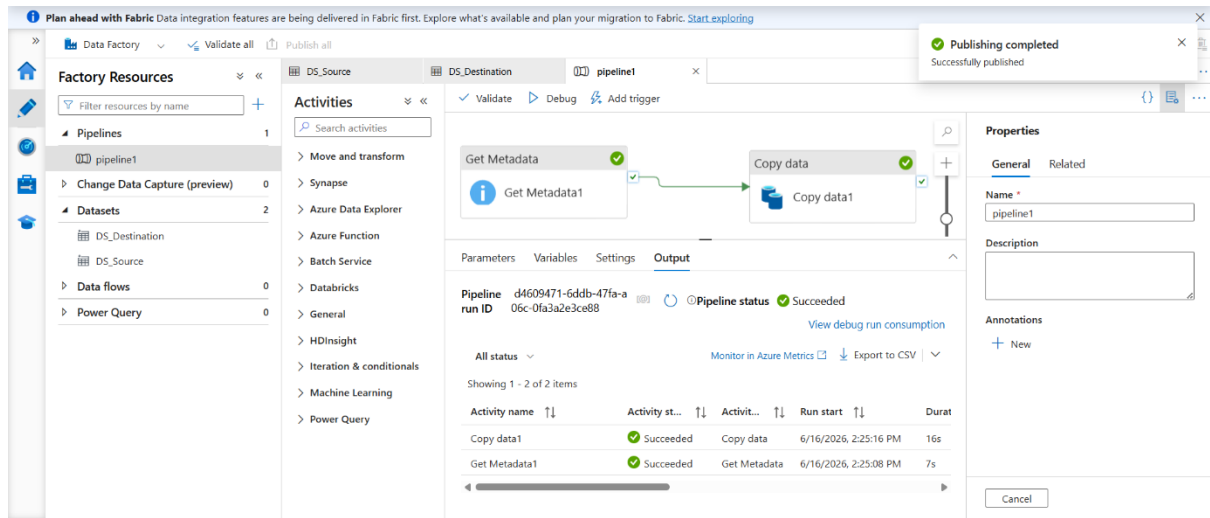


Fig 13: The debug execution output confirms a "Succeeded" status for the overall pipeline, verifying that both the metadata validation and the data copy activities ran flawlessly without any errors.

Task 6: IAM Roles

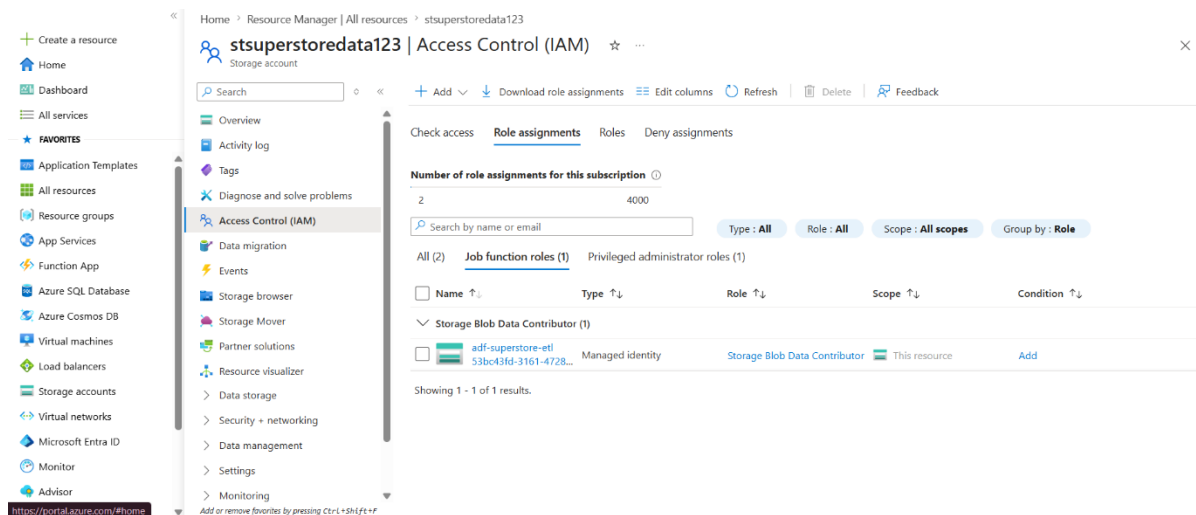


Fig 14: The Storage Account's Access Control (IAM) interface verifies that the Azure Data Factory's Managed Identity has been successfully granted the "Storage Blob Data Contributor" role, ensuring secure and authorized data transfer capabilities.